School of Future Tech

Case Study Report

on

BUDGET HANDLER

by

Rima Maji

Index

1. Introduction to the Case Study

Managing personal expenses is an important part of financial planning. Many students and young professionals struggle to separate essential expenses from non-essential spending. This project, “Need & Want Budget Tracker,” is a simple React.js based web application developed to help users categorize their expenses into two sections: “Needs” and “Wants.”

The application allows users to enter the expense name, amount, and category. Based on the selected category, the expense is displayed in the corresponding list. The system also validates user input and provides feedback messages.

This project demonstrates the practical use of React concepts such as components, hooks, state management, event handling, conditional rendering, and CSS styling.

---

2. Problem Statement / Case Background (Abstract)

In day-to-day life, people often spend money without tracking whether the expense is necessary or optional. This can lead to poor budgeting habits and unnecessary expenses.

The main objective of this project is to create a lightweight and user-friendly budget tracking system where users can:

- Add expenses easily.
- Classify expenses into “Needs” and “Wants.”
- View categorized expenses separately.
- Improve budgeting awareness.
- Practice financial discipline.

The application is built using React.js and provides an interactive frontend interface with real-time updates.

---

3. Problem Statement / Case Study Design

System Design

The application follows a component-based frontend architecture using React.js.

Input Section

The user enters:

- Expense Name
- Expense Amount
- Expense Category (Need or Want)

Processing Section

The application validates:

- Empty fields
- Invalid amount values
- Category selection

After validation:

- The expense object is created.
- The data is stored in state variables.
- Expenses are displayed dynamically.

Output Section

Two separate sections are displayed:

- Needs List
- Wants List

Each list contains:

- Expense Name
- Expense Amount

Workflow of the System

1. User fills the form.
2. User selects category.

3. User clicks Add Expense.
4. Validation is performed.
5. Expense is stored in the correct category.
6. Lists update automatically.

---

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Technologies Used

Technology	Purpose
React.js	Frontend framework
JavaScript	Application logic
CSS	Styling and UI design
Vite	Fast React development environment
HTML	Structure of webpage

React Concepts Used

useState Hook

The project uses React's `useState()` hook for managing dynamic data.

Example:

const [needs, setNeeds] = useState([]);

const [wants, setWants] = useState([]);

Event Handling

The form submission is managed using an event handler function.

function addExpense(e) {

e.preventDefault();

}

Conditional Logic

The application checks the selected category and stores the expense in the correct list.

if (category === "Need") {

setNeeds([...needs, newExpense]);

} else {

setWants([...wants, newExpense]);

}

Validation Algorithm

The application validates:

- Empty input fields
- Invalid amounts
- Missing category selection

Validation Flow

1. Check if fields are empty.
2. Check if amount is greater than zero.
3. If validation fails → show error message.
4. If validation passes → add expense.

5. Problem Statement / Case Study Implementation Details and Snapshots

Main Features Implemented

Expense Input Form

The user can:

- Enter expense name
- Enter amount
- Select Need or Want category

Dynamic Expense Lists

The application updates the lists instantly without refreshing the page.

Error Handling

The app shows messages for:

- Empty fields
- Invalid amounts
- Successful addition of expenses

Important Code Snippet

const newExpense = {

id: Date.now(),

name: expenseName,

amount: expenseAmount,

};

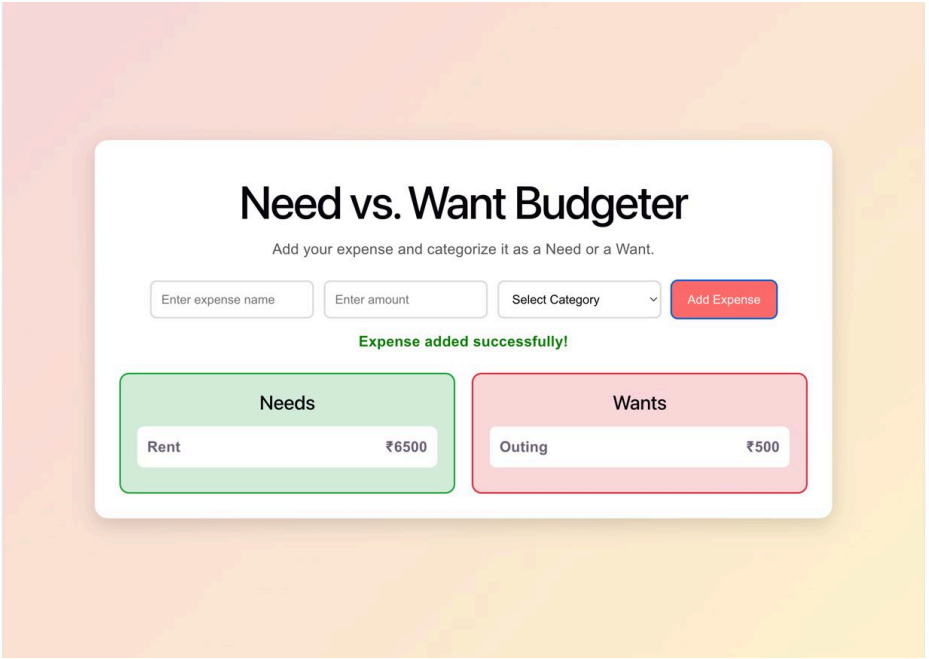
This creates a unique expense object for every entry.

Styling Details

The application uses CSS for:

- Form layout
- Buttons
- Expense cards
- Responsive alignment
- Separate sections for Needs and Wants

Snapshots



---

6. Problem Statement / Case Study Results and Conclusion

Results

The project was successfully implemented using React.js. The application correctly:

- Accepts user expense inputs.
- Categorizes expenses into Needs and Wants.
- Performs validation checks.
- Displays expenses dynamically.

- Provides a simple and user-friendly interface.

Advantages of the System

- Easy to use
- Beginner-friendly interface
- Helps users understand budgeting
- Real-time updates using React state management
- Lightweight and fast application

Conclusion

The “Need & Want Budget Tracker” project demonstrates how React.js can be used to build interactive and dynamic web applications. The project successfully solves the problem of expense categorization and basic budgeting.

This application can be further enhanced by adding:

- Local storage/database support
- Expense deletion and editing
- Monthly budget limits
- Charts and analytics
- Authentication system
- Mobile responsiveness

Overall, the project is a good example of frontend development using React.js and showcases important concepts such as state management, event handling, validation, and dynamic rendering.
